ACTUALIZACIÓN DE ENTREGA / 11 DE NOVIEMBRE DE 2026

Backend y evidencia de entrega

Complemento de los documentos originales de NutriBalance. Escenario 2: NutriTurnos.

Qué actualiza este anexo

Los cinco PDF aportados por el equipo se conservan sin cambios. Este anexo agrega la inspección del backend y de PostgreSQL que no estaba cubierta por los documentos elaborados desde el frontend. No presenta una nueva auditoría de Lighthouse ni afirma una publicación realizada desde esta PC.

Backend preparado para el frontend existente

La API mantiene **/api/v1**, los identificadores UUID y los campos del contrato. El acceso usa la cookie **nutri.sid**, sesiones persistidas en PostgreSQL y **X-CSRF-Token**. El frontend conserva su gateway y no se modificó su código.

El perfil de túnel configura **127.0.0.1**, confianza exclusiva en el proxy local y cookie **HttpOnly / Secure / SameSite=Lax**. El origen permitido es **https://nutribalance-mauricio.maauri.chatgpt.site**. El hostname **api.neadev.com** debe seguir dirigido al Express de la PC donde ya se ejecuta Cloudflare Tunnel.

Verificaciones nuevas

Control	Resultado
TypeScript y compilación de Express	Aprobados; cliente Prisma generado.
Pruebas unitarias	7 aprobadas.
Integración con PostgreSQL real	23 aprobadas sobre nutri_test local separada.
Total de la suite del backend	30 pruebas aprobadas, sin fallos.
Arranque portable	Base, dos migraciones, seed y servidor compilado operativos.
HTTP real del servidor compilado	Health 200; anónimo 401; CSRF/login/me 200; logout 204 y luego 401.

Las pruebas incluyen reservas simultáneas, cancelación y liberación de horario, pacientes, historial, sesiones, CSRF y el guardado atómico de consulta con turno atendido. La prueba de fallo deliberado verifica rollback; el error interno que produce forma parte de ese caso aprobado.

ESQUEMA REAL / POSTGRESQL

Modelo físico verificado

Fuente: backend/prisma/schema.prisma y las migraciones 202610190001_inicial y 202610190002_integridad. Estos archivos se incluyen con el código del backend.

Tabla	Campos y restricciones comprobados
patients	id UUID PK; first_name, last_name y phone; document_number opcional y único; birth_date DATE; email, background, allergies y medication; created_at y updated_at.
appointments	id UUID PK; patient_id FK opcional; contact_name, contact_phone y contact_email; starts_at y ends_at TIMESTAMPTZ; status enum; timestamps. Duración exacta de 30 minutos y un solo turno activo por starts_at.
consultations	id UUID PK; patient_id FK obligatorio; appointment_id FK opcional y único; date DATE; reason, objective, observations e instructions; weight_kg, height_cm y waist_cm DECIMAL(5,2); timestamps.
users	id UUID PK; email único; display_name; password_hash Argon2id; role; created_at. No se publican contraseñas ni hashes en la API.
sessions	sid PK; sess JSON; expire con índice. Guarda la sesión del servidor. El usuario se referencia en su JSON, sin afirmar una FK SQL que no existe.

Relaciones y reglas

Paciente - turno: un paciente puede tener cero o muchos turnos. Una solicitud pendiente puede no tener ficha; confirmar exige vincular un paciente existente.

Paciente - consulta: cada consulta pertenece a una ficha; cada ficha puede tener cero o muchas consultas.

Turno - consulta: una consulta puede no tener turno; un turno admite como máximo una consulta vinculada.

Integridad: FK con eliminación restringida, documento único opcional, controles de duración y rangos de medidas. Los turnos cancelados liberan el horario.

Transacción: al guardar una consulta vinculada válida, la API cambia el turno confirmado a atendido dentro de la misma transacción. Un fallo revierte ambos cambios.

El modelo lógico original de la entrega es coherente con estos recursos. Su advertencia sobre SQL no inspeccionado queda ampliada por esta verificación. D1/SQLite pertenece al respaldo del frontend y no representa la PostgreSQL activa.

Arranque y demostración

1. Actualizar y preparar

Actualizar el repositorio backend, entrar en **backend** e instalar con **npm ci** usando Node.js 24. Conservar el .env privado y la base existentes de esa PC.

2. Iniciar la API con el perfil del túnel

Usar **npm run serve:portable:tunnel** para preparar y ejecutar PostgreSQL portable más la API. Si PostgreSQL ya está administrado por Docker u otro servicio, usar **npm run serve:tunnel**. El comando conserva las contraseñas de cuentas existentes y solo crea la cuenta ficticia si falta.

3. Mantener la ruta existente de Cloudflare

El hostname **api.neadev.com** debe apuntar a **http://127.0.0.1:3000**, sin agregar /api/v1. El navegador consume la URL HTTPS pública. Si se cambia PORT, actualizar el destino del túnel. Sus tokens y credenciales permanecen fuera del repositorio y del ZIP.

4. Comprobar el flujo público

Ejecutar **npm run check:public**. Luego demostrar con datos ficticios: solicitar turno, ingresar, crear o buscar ficha, confirmar el turno, registrar consulta y recuperar el historial al recargar. Una consulta vinculada requiere que el horario del turno ya haya comenzado. Cerrar sesión al terminar.

Documentación seleccionada

La carpeta docs/entrega contiene análisis funcional, wireframes/prototipo, modelo lógico, evidencias de SEO, memoria final y este anexo. docs/evidencias contiene los reportes Lighthouse originales HTML/JSON y la captura disponible. El README de entrega vincula cada punto de la consigna con su archivo.

Atribución y límites actuales

La distribución declarada se mantiene: Mauricio en frontend y Bruno en backend/base de datos. En esta revisión posterior, **Codex** apoyó la preparación del perfil de túnel, los comandos de arranque, nuevas pruebas y la organización de la entrega. Esto amplía la declaración de IA de la memoria original, sin atribuirse la implementación original de Bruno.

La publicación en api.neadev.com se levanta en otra PC y no se realizó desde esta. Debe comprobarse antes de la defensa. Tampoco se cambiaron los defectos ni las diferencias entre código y publicación del frontend detectados en la revisión anterior; corresponden a su responsable. La defensa oral y el acceso del docente a ambos repositorios siguen siendo verificaciones externas.

Referencias

Backend: github.com/Deathstone16/NutriBalanceExpress
Frontend: github.com/Maaupelozo/nutribalance-tp-web
Web pública: nutribalance-mauricio.maauri.chatgpt.site